6 CHAPTER 6: IMPLEMENTATION

6.1 Introduction

The implementation phase of the USIU Campus Navigator mobile application represented the crucial transformation of theoretical design specifications and comprehensive requirements documentation into a fully operational, production-ready Android application. This extensive development stage encompassed the systematic creation of multiple interconnected modules, seamless system integration across various Android components, and rigorous testing on multiple device configurations. The primary objective was to deliver a robust, intuitive, and feature-complete navigation solution that would serve the diverse needs of United States International University Africa's academic community.

The implementation prioritized creating an accessible platform where students, faculty, staff, and campus visitors could efficiently navigate the extensive USIU campus facilities, locate specific buildings and services with precision, access critical emergency contact information instantly, and personalize their experience through persistent favorites management. Special emphasis was placed on developing a responsive user interface that maintains consistency across various device sizes and orientations while adhering to modern Material Design principles and USIU's official branding guidelines.

The comprehensive feature set includes advanced Google Maps integration with campus-boundary restrictions, real-time GPS location tracking with accuracy indicators, intelligent facility status systems that dynamically reflect operational hours, and secure local data persistence for user preferences. The application architecture was meticulously designed to serve as an indispensable daily tool for both newcomers orienting themselves to the campus layout and long-term community members seeking optimized navigation between frequently visited locations, thereby significantly enhancing overall campus accessibility and user experience.

6.2 Frontend Implementation

The frontend architecture of the USIU Campus Navigator was constructed using Java programming language within the Android native development framework, leveraging the full capabilities of the Android SDK to deliver a polished, responsive, and highly intuitive user interface. The application systematically implements Material Design 3 components throughout

all interface elements to ensure visual consistency, accessibility compliance, and professional aesthetics that align with modern mobile application standards. The carefully structured interface facilitates effortless navigation through four logically organized main sections via a persistent bottom navigation bar, enabling seamless transitions between real-time interactive mapping, comprehensive facility exploration, personalized favorites management, and critical emergency contact access.

The design philosophy consistently emphasizes usability while maintaining strict adherence to USIU's official brand identity, incorporating the university's signature color palette of dark blue (#002147) and accent gold (#CFB991) throughout all visual elements. This strategic implementation ensures a cohesive and institutionally aligned user experience across all device configurations, from compact mobile phones to larger tablet displays, while maintaining optimal performance and responsiveness.

i. Main Application Structure and Navigation Implementation

The Map Interface serves as the foundational component of the USIU Campus Navigator, acting as the primary entry point for users seeking spatial orientation. Implemented within the MapFragment class, this interface leverages the Google Maps SDK for Android to render a high-fidelity, vector-based map of the campus. The fragment implements the OnMapReadyCallback interface, ensuring that all map-related logic, such as setting camera angles and enabling location layers, is executed asynchronously only after the Google Play Services have fully rendered the map tiles. This approach prevents the main UI thread from hanging during initialization, resulting in a stutter-free user experience.

To ensure the application remains focused on its primary objective, campus navigation, a strict Camera Boundary Restriction was implemented. Without this constraint, users could accidentally scroll away from the campus into surrounding neighborhoods like Roysambu or Garden Estate, losing their orientation. The implementation defines a specific LatLngBounds object corresponding to the geographical corners of the USIU Africa campus. The camera logic is programmed to reject any scroll or pan gestures that would move the viewport outside these defined coordinates, effectively "locking" the user's view to the relevant educational environment.

A significant portion of the implementation logic is dedicated to Real-Time Facility Status. To solve the problem of students walking to closed facilities, the MapFragment initializes a status tracker upon creation. This logic utilizes the Java Calendar instance to retrieve the current day of the week and hour of the day. It then applies conditional logic—specific to USIU operations—to determine if facility types like "Office,"

"Restaurant," or "Pool" are currently open. For example, the logic dictates that the pool is only open on weekdays between 7 AM and 5 PM, while academic buildings remain accessible for longer durations.

The interface also integrates Quick Access Navigation through card interaction listeners. Instead of requiring users to manually pinch and zoom to find popular locations, the implementation attaches OnClickListener events to specific UI cards (Library, Sports, Cafeteria). When a user taps a card, the application programmatically triggers a CameraUpdateFactory.newLatLngZoom command. This smoothly animates the camera to the precise latitude and longitude of the selected facility, adjusting the zoom level to 17.0f to provide an optimal view of the building's footprint.

Finally, the MapFragment handles the critical Runtime Permission Logic required for the "My Location" layer. Before enabling the blue dot that represents the user's GPS position, the code checks if ACCESS_FINE_LOCATION has been granted by the user. If the permission is missing, the app requests it via ActivityCompat; if granted, the mMap.setMyLocationEnabled(true) method is called. This ensures the app complies with Android's privacy standards while providing the essential wayfinding feature that allows students to see their position relative to campus landmarks.

```java
public class MapFragment extends Fragment implements OnMapReadyCallback {
    private GoogleMap mMap;
    1 usage
    private FusedLocationProviderClient fusedLocationClient;
    2 usages
    private static final int LOCATION_PERMISSION_REQUEST_CODE = 1;

    // USIU Campus Boundaries - Expanded for more zoom out
    1 usage
    private static final LatLngBounds USIU_BOUNDS = new LatLngBounds(
            new LatLng( latitude: -1.2160,  longitude: 36.8760), // Expanded Southwest corner
            new LatLng( latitude: -1.2110,  longitude: 36.8820)  // Expanded Northeast corner
    );

    // Facility status tracker
    6 usages
    private Map<String, String> facilityStatus;

    public MapFragment() {
        // Required empty public constructor
    }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_map, container,  attachToRoot: false);

        // Initialize facility status
        initializeFacilityStatus();

        // Initialize map
```

Figure 23: MapFragment Class Declaration and Geographical Constants

Figure 23 above illustrates the foundational setup of the map interface within the MapFragment class. The code snippet highlights the declaration of essential class-level variables, including the GoogleMap object and the FusedLocationProviderClient for GPS tracking. Additionally, it shows the definition of the static USIU_BOUNDS constant, which establishes the strict geographical coordinates used to restrict the camera view to the campus area, ensuring users remain focused on the relevant environment..

Lines 30–32: Declares essential class-level variables. mMap holds the reference to the visual map, while fusedLocationClient is the entry point for the Google Play Services location APIs, used to retrieve the device's precise GPS coordinates.

Lines 35–38: Defines USIU_BOUNDS as a static constant using the LatLngBounds class. This block hardcodes the specific Southwest and Northeast geographical coordinates of the USIU campus, creating the mathematical rectangle used to constrain the user's view.

Line 41: Initializes the declaration for facilityStatus. This Map<String, String> data structure is designed to store the real-time operational state (e.g., "Open" or "Closed") of various campus facilities, decoupling the data storage from the UI logic.

```
28  public class MapFragment extends Fragment implements OnMapReadyCallback {
48      public View onCreateView(LayoutInflater inflater, ViewGroup container,
50          View view = inflater.inflate(R.layout.fragment_map, container, attachToRoot: false);
51
52          // Initialize facility status
53          initializeFacilityStatus();
54
55          // Initialize map
56          initializeMap();
57
58          // Initialize location client
59          fusedLocationClient = LocationServices.getFusedLocationProviderClient(requireActivity());
60
61          // Set up button listeners
62          setupButtonListeners(view);
63
64          return view;
65      }
66
        1 usage
67      private void initializeFacilityStatus() {
68          facilityStatus = new HashMap<>();
69          Calendar calendar = Calendar.getInstance();
70          int dayOfWeek = calendar.get(Calendar.DAY_OF_WEEK);
71          int hour = calendar.get(Calendar.HOUR_OF_DAY);
72
73          // Office hours: 8AM-5PM Monday-Friday
74          boolean isOfficeOpen = (dayOfWeek >= Calendar.MONDAY && dayOfWeek <= Calendar.FRIDAY) &&
75                  (hour >= 8 && hour < 17);
76
77          // Restaurant hours: Close at 8PM daily
```

Figure 24: Fragment Initialization and Real-Time Status Logic

Figure 24 above illustrates the initialization logic executed when the map fragment is created. The code within the onCreateView method demonstrates how the application prepares the user interface by inflating the layout and calling helper methods to set up the facility status tracking. This section is critical for integrating the backend logic with the visual map component, ensuring that features like the search bar and smart status indicators are ready for user interaction immediately upon launch**.**

Lines 48–62: The onCreateView method orchestrates the fragment's startup. Rather than cluttering the main lifecycle method, it delegates tasks to helper methods: initializeFacilityStatus() prepares the data, initializeMap() loads the visual component, and setupButtonListeners() attaches click events to the UI.

Lines 67–71: The initializeFacilityStatus method begins the real-time logic. It creates a Java.util.Calendar instance to capture the current system time (Day of Week and Hour of Day), which is the foundation for deciding if a facility is currently open.

Lines 74–75: Implements the specific business logic for administrative offices. It evaluates a boolean expression checking if the current day is between Monday and Friday (Calendar.MONDAY to Calendar.FRIDAY) and if the current time is within standard working hours (08:00 to 17:00).

```
no usages
133 @Override
134 public void onMapReady(GoogleMap googleMap) {
135     mMap = googleMap;
136
137     // Set map restrictions first
138     setupMapRestrictions();
139
140     // Check for location permissions
141     if (ActivityCompat.checkSelfPermission(requireContext(),
142             Manifest.permission.ACCESS_FINE_LOCATION) != PackageManager.PERMISSION_GRANTED) {
143
144         ActivityCompat.requestPermissions(requireActivity(),
145                 new String[]{Manifest.permission.ACCESS_FINE_LOCATION},
146                 LOCATION_PERMISSION_REQUEST_CODE);
147         return;
148     }
149
150     // Enable current location
151     mMap.setMyLocationEnabled(true);
152
153     // Set up map - NO MARKERS ADDED
154     setupMap();
155 }
156
1 usage
157 private void setupMapRestrictions() {
158     // Restrict camera to expanded USIU campus area
159     mMap.setLatLngBoundsForCameraTarget(USIU_BOUNDS);
160     mMap.setMinZoomPreference(15.0f); // Reduced from 16.0f for more zoom out
```

Figure 25: Asynchronous Map Setup and Runtime Permission Handling.

Figure 25 illustrates the initialization process within the MapFragment class, specifically focusing on the onMapReady callback and the runtime permission handling logic. This method serves as the entry point for all map-related operations, ensuring that the Google Map object is fully loaded before any interactions occur. Additionally, the code demonstrates a critical security check where the application verifies if the user has granted the ACCESS_FINE_LOCATION permission; if the permission is missing, the system gracefully handles the scenario by pausing GPS-dependent features to prevent runtime crashes

Lines 134–136: The onMapReady callback is triggered asynchronously once the Google Maps service is fully loaded. It initializes the global mMap variable, allowing the rest of the class to interact with the map interface.

Lines 141–148: Implements critical Android security logic. Before accessing the GPS, the code uses checkSelfPermission to verify if ACCESS_FINE_LOCATION has been granted. If the permission is missing, it pauses execution to request it from the user via requestPermissions, preventing the app from crashing due to security violations.

Lines 157–160: The setupMapRestrictions method acts as a "digital fence." It applies setLatLngBoundsForCameraTarget(USIU_BOUNDS) to strictly limit the scrollable area to the campus. It also sets a MinZoomPreference of 15.0f, which prevents users from zooming out too far and losing context of the campus details.

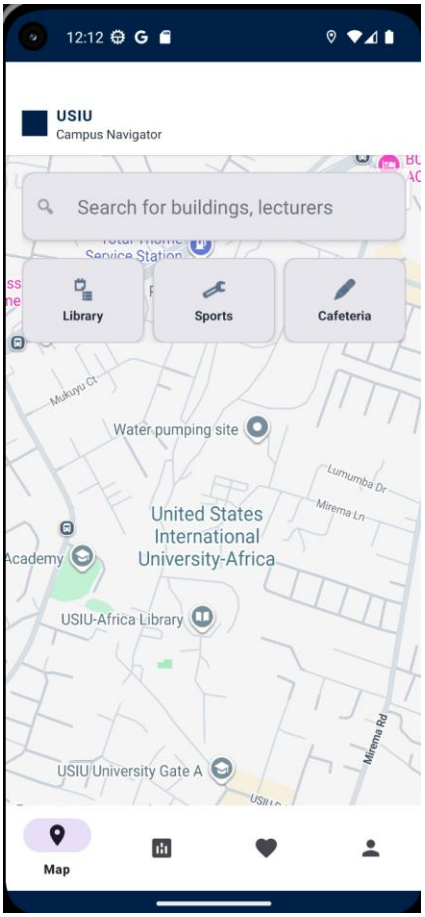

Figure 26: Map Interface showing the Google Map

Figure 26 above illustrates a screenshot of the Map Interface showing the Google Map, the Search Bar at the top, and the Quick Access Cards (Library, Sports, Cafeteria) at the bottom

The Map Interface is designed as the primary navigation hub, prioritizing immediate spatial orientation and ease of access through a layered UI approach. Dominating the screen is the central map canvas, which renders high-fidelity vector tiles of the USIU campus. This dynamic layer supports standard gesture interactions, allowing users to pinch-to-zoom for granular views of building footprints or rotate the camera to align with their physical orientation. The visual style is customized to highlight campus footpaths and structures in high contrast, while the user's real-time position is tracked via a pulsating blue dot with a directional cone, providing immediate context regarding their location relative to their surroundings.

Situated at the top of the interface is a floating Search Bar, designed with rounded corners and elevation consistent with Material Design 3 card standards. This widget acts as the primary entry point for specific queries; tapping it expands the view to allow text input for building names or facility categories. To the right of the search field, a filter icon provides users with the ability to toggle the visibility of specific marker types, such as "Academic" or "Dining," effectively reducing visual clutter on the map and allowing for a more focused navigation experience.

Anchored at the bottom of the screen is a horizontal "Quick Access" carousel, implemented as a scrollable list of cards overlaying the map view. Each card displays a high-quality thumbnail and the name of a major facility, such as the Library or Sports Complex. This feature addresses the common user challenge of not knowing exactly what to search for upon first launch. Tapping any card in this carousel triggers a programmatic camera update, smoothly flying the viewport to the selected location. Just above this carousel sits the "My Location" Floating Action Button (FAB), which allows users to instantly re-center the map on their GPS coordinates if they have panned away to explore other areas of the campus.

ii. Facilities Directory Implementation

The Facilities Directory serves as the central information hub of the application, providing users with a comprehensive, searchable list of all key locations within the USIU campus. Unlike the Map Interface, which offers spatial context, the Directory is optimized for quick data retrieval and status checking. Implemented within the FacilitiesFragment, this module utilizes a RecyclerView

component to render a scrollable, high-performance list of facility objects. The architecture separates the data layer from the presentation layer using a custom FacilitiesAdapter, which binds raw data, such as building names, categories, and operating hours to the visual layout defined in the XML card files. This separation of concerns ensures that the list can scale efficiently to accommodate hundreds of entries without causing UI lag or memory leaks.

A core innovation within this module is the "Smart Status" logic, designed to address the common frustration of students walking to closed facilities. The adapter does not simply display static operating hours; instead, it performs a real-time calculation every time a card is bound to the view. By capturing the device's current system time and comparing it against the specific operating window of each facility type (e.g., Academic, Dining, or Administration), the system dynamically assigns a status of "Open" or "Closed." This status is visually reinforced through color-coded indicators Gold for open and Red for closed allowing users to assess availability at a glance while scrolling, significantly enhancing the utility of the directory.

To further improve usability, the Facilities Directory incorporates a robust search and filtering mechanism. A SearchView widget located at the top of the interface allows users to filter the list in real-time by typing keywords, such as "Library" or "Cafeteria." The implementation logic uses a filtering algorithm within the adapter to instantly update the data set, removing irrelevant items without reloading the entire fragment. This responsiveness is crucial for new students who may know the name of a building but not its category or location. Additionally, tapping on any facility card triggers a navigation intent, linking the directory directly back to the Map Interface to show the selected location.

```
23  public class FacilitiesAdapter extends RecyclerView.Adapter<FacilitiesAdapter.FacilityViewHolder> {
46
47      @Override
48      public void onBindViewHolder(@NonNull FacilityViewHolder holder, int position) {
49          Facility facility = facilityList.get(position);
50
51          holder.tvFacilityName.setText(facility.getName());
52          holder.tvCategory.setText(facility.getCategory());
53          holder.tvFacilityDescription.setText(facility.getDescription());
54          holder.tvStatus.setText(facility.getStatus());
55
56          // Set status color - YELLOW TEXT ONLY, NO BACKGROUND
57          if ("Open".equals(facility.getStatus())) {
58              holder.tvStatus.setTextColor(holder.itemView.getContext().getColor(R.color.usiu_status_open)); // Yellow text
59              holder.tvStatus.setBackgroundColor(holder.itemView.getContext().getColor(android.R.color.transparent)); // No background
60          } else {
61              holder.tvStatus.setTextColor(holder.itemView.getContext().getColor(R.color.usiu_warning_red)); // Red text
62              holder.tvStatus.setBackgroundColor(holder.itemView.getContext().getColor(android.R.color.transparent)); // No background
63          }
64
65          // Set favorite button state
66          if (isFacilityFavorite(facility)) {
67              holder.btnFavorite.setTextColor(holder.itemView.getContext().getColor(R.color.usiu_warning_red));
68              holder.btnFavorite.setBackgroundResource(R.drawable.button_pill_active);
69          } else {
70              holder.btnFavorite.setTextColor(holder.itemView.getContext().getColor(R.color.usiu_primary_blue));
71              holder.btnFavorite.setBackgroundResource(R.drawable.button_pill_inactive);
72          }
73
74          // Set up button click listeners
75          setupButtonListeners(holder, facility);
```

Figure 27 : Facilities directory implementation

Figure 27 presents the visual logic used to implement the "Smart Status" indicators within the directory list. The code snippet details the conditional statements that determine the color and text of the status pill based on the facility's real-time availability. If the getStatus() method returns "Open," the text is rendered in usiu_status_open (Gold); otherwise, it shifts to usiu_warning_red (Red). This immediate visual feedback mechanism is essential for allowing users to quickly assess facility accessibility while scrolling through the directory.

Lines 48-50: The onBindViewHolder method is the core logic block for the list. It retrieves the Facility object corresponding to the current scroll position from the facilityList.

Lines 56-61: Implements the visual logic for the status indicator. If the facility status matches "Open", the text color is set to usiu_status_open (Yellow) with a transparent background. Conversely, if closed, the text color shifts to usiu_warning_red (Red), providing immediate visual feedback to the user.

```
23  public class FacilitiesAdapter extends RecyclerView.Adapter<FacilitiesAdapter.FacilityViewHolder> {
76      }
77
        1 usage
78      private void setupButtonListeners(FacilityViewHolder holder, Facility facility) {
79          // Favorite button - toggles between red and blue
80          holder.btnFavorite.setOnClickListener( View v -> {
81              if (isFacilityFavorite(facility)) {
82                  // Remove from favorites
83                  removeFromFavorites(facility);
84                  holder.btnFavorite.setTextColor(holder.itemView.getContext().getColor(R.color.usiu_primary_blue));
85                  holder.btnFavorite.setBackgroundResource(R.drawable.button_pill_inactive);
86              } else {
87                  // Add to favorites
88                  addToFavorites(facility);
89                  holder.btnFavorite.setTextColor(holder.itemView.getContext().getColor(R.color.usiu_warning_red));
90                  holder.btnFavorite.setBackgroundResource(R.drawable.button_pill_active);
91              }
92          });
93
94          // Directions button - opens Google Maps
95          holder.btnDirections.setOnClickListener( View v -> {
96              Uri gmmIntentUri = Uri.parse( uriString: "google.navigation:q=USIU+Africa,+Nairobi");
97              Intent mapIntent = new Intent(Intent.ACTION_VIEW, gmmIntentUri);
98              mapIntent.setPackage("com.google.android.apps.maps");
99              holder.itemView.getContext().startActivity(mapIntent);
100         });
101
102         // Call button - opens dialer
103         holder.btnCall.setOnClickListener( View v -> {
104             Intent intent = new Intent(Intent.ACTION_DIAL);
```

Figure 28 : Facilities directory implementation

Figure 28 showcases the implementation of interactive action buttons within the facility card, specifically the "Favorite" and "Directions" features. The code demonstrates how the application manages user preferences by checking the local SharedPreferences via the isFacilityFavorite helper method; if a facility is bookmarked, the heart icon's state is updated to reflect this status visually. Furthermore, the snippet reveals the intent logic attached to the "Directions" button, which constructs a specific Uri to launch external navigation apps, seamlessly integrating the campus directory with broader mapping services.

Lines 64-67: Checks the local persistence layer (SharedPreferences) via the helper method isFacilityFavorite. If the facility is saved as a favorite, the heart icon button color changes to red and the background drawable is updated to button_pill_active, indicating the active state.

Lines 93-98: Sets up the "Directions" button listener. This code constructs a Uri with the navigation query for USIU Africa and launches an Intent specifically targeting the Google Maps application (com.google.android.apps.maps), allowing the user to get turn-by-turn navigation.

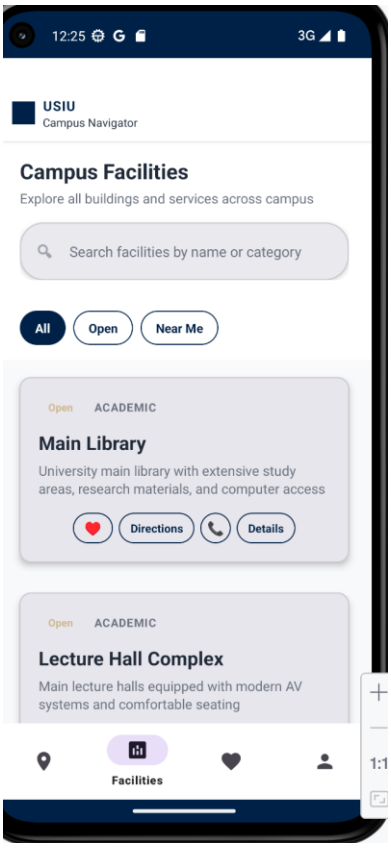

Figure 29:Facilities Directory UI

Figure 29 above demonstrates the facilities directory UI. Below the primary text information, each card features a row of interactive action buttons. The Favorite button allows users to toggle the saved state of the facility, providing a personalized experience. The Directions button triggers an external map intent, while the Call button connects directly to the facility's office extension. Finally, a Details button is available to expand a bottom sheet containing comprehensive information, such as the full description and operating history.

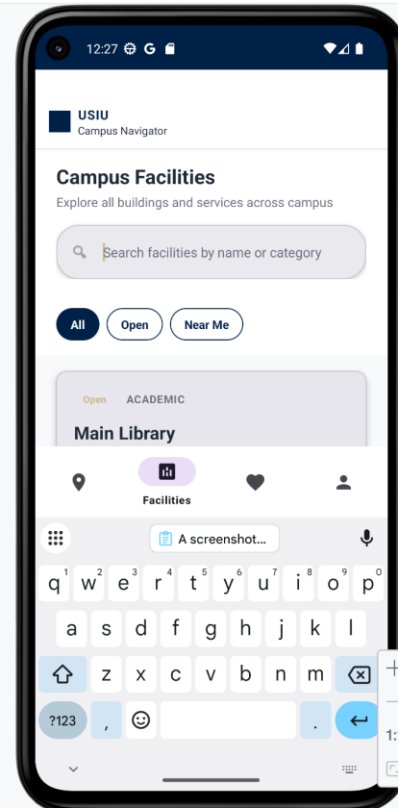

Figure 30: Facilities Directory UI

Figure 30 above illustrates a screenshot of the Facilities Directory showing the list of cards with the Yellow/Red status text and the action buttons.

The Facilities Directory UI is designed as a vertical, scrollable list of information cards. Each card serves as a compact summary of a specific campus location, displaying the Facility Name (tvFacilityName) and Category (tvCategory) in bold, legible typography. The status of the building is prominently displayed on the right side of the card; as detailed in the code, this text appears in Yellow when the facility is open and Red when closed, allowing users to scan the list rapidly for available services without opening detailed views.

This layout maximizes functionality while maintaining a clean, uncluttered aesthetic consistent with USIU branding.

iii. Profile and Support UI

The Profile and Support UI is a critical component designed to serve as the application's safety and communication hub. While the Map and Directory fragments focus on exploration, the Profile

fragment is engineered for immediate access to institutional support services. Implemented within the ProfileFragment class, this interface aggregates essential contact points—such as Campus Security, Health Services, and the Main Office—into a clean, accessible layout. The logic behind this fragment prioritizes speed and reliability; rather than navigating through complex menus, users are presented with direct-action cards that trigger native system functions immediately upon interaction.

Technically, this module relies heavily on Android's Implicit Intents system to bridge the gap between the application and the device's core telephony and messaging capabilities. The implementation handles OnClickListener events for various UI cards, programmatically constructing Intent objects with specific actions like ACTION_DIAL for phone calls and ACTION_SENDTO for emails. This approach ensures that the application does not need to reinvent communication tools but instead leverages the user's preferred dialer and email clients, ensuring a familiar and reliable user experience.

Additionally, the Profile fragment serves as a secondary navigation anchor. It includes logic to communicate back to the central MainActivity, allowing users to jump directly from the contact screen back to the map view if they need to locate the office they just contacted. Error handling is also integrated into the communication logic; specifically for email intents, the code wraps the start activity call in a try-catch block. This defensive programming ensures that the app remains stable and provides helpful feedback (via Toasts) even if the user's device lacks a configured email client.

```
14      public class ProfileFragment extends Fragment {
30          private void setupClickListeners(View view) {
32              view.findViewById(R.id.cardSecurity).setOnClickListener( View v -> {
33                  Intent intent = new Intent(Intent.ACTION_DIAL);
34                  intent.setData(Uri.parse( uriString: "tel:+254730116111"));
35                  startActivity(intent);
36              });
37
38              // Health Services - Direct dial
39              view.findViewById(R.id.cardHealth).setOnClickListener( View v -> {
40                  Intent intent = new Intent(Intent.ACTION_DIAL);
41                  intent.setData(Uri.parse( uriString: "tel:+254730116080"));
42                  startActivity(intent);
43              });
44
45              // Main Office - Direct dial
46              view.findViewById(R.id.cardMainOffice).setOnClickListener( View v -> {
47                  Intent intent = new Intent(Intent.ACTION_DIAL);
48                  intent.setData(Uri.parse( uriString: "tel:+254730116000"));
49                  startActivity(intent);
50              });
51
52              // Open Map button - Navigate to Map screen
53              view.findViewById(R.id.btnOpenMap).setOnClickListener( View v -> {
54                  // Navigate to Map tab
55                  if (getActivity() instanceof MainActivity) {
56                      ((MainActivity) getActivity()).navigateToMap();
57                  }
58              });
59
60              // Email button - Open email app
```

Figure 31 : Implementation of Direct Dial Intents and Internal Navigation Logic.

Figure 31 illustrates the implementation of Android Implicit Intents within the ProfileFragment to facilitate direct communication features. The code snippet focuses on the setupClickListeners method, where specific ACTION_DIAL intents are attached to contact cards like "Campus Security." By parsing the telephone number into a URI, the application can hand off the dialing process to the system's native phone app. This implementation is crucial for the "Emergency Contacts" feature, providing users with a reliable, one-tap solution for urgent situations without requiring manual number entry.

Lines 30–36: The setupClickListeners method initializes the user interaction logic. Specifically, lines 32–36 attach an OnClickListener to the Security card. When tapped, it creates an **Implicit Intent** with the ACTION_DIAL action. By parsing the telephone string (tel:+254...), the app hands off the operation to the Android system's default dialer, pre-filling the emergency number without requiring the user to type it.

Lines 53–57: This block handles internal app navigation rather than external intents. When the "Open Map" button is clicked, the code checks if the hosting activity is an instance of MainActivity. If true, it casts the activity context and calls the public method MapsToMap(). This allows the Fragment to communicate upward to the Activity to switch tabs or fragments programmatically.

```
56                 ((MainActivity) getActivity()).navigateToMap();
57             }
58         });
59
60         // Email button - Open email app
61         view.findViewById(R.id.btnEmail).setOnClickListener( View v -> {
62             Intent intent = new Intent(Intent.ACTION_SENDTO);
63             intent.setData(Uri.parse( uriString: "mailto:info@usiu.ac.ke"));
64             intent.putExtra(Intent.EXTRA_SUBJECT,  value: "USIU Campus Navigator Inquiry");
65             try {
66                 startActivity(intent);
67             } catch (Exception e) {
68                 Toast.makeText(getContext(),  text: "No email app found", Toast.LENGTH_SHORT).show();
69             }
70         });
71     }
72 }
```

Figure 32 : Configuration of Email Intent with Runtime Exception Handling.

Figure 32 demonstrates the configuration of the ACTION_SENDTO intent used for the "Email Us" feature. The code shows how the application constructs a highly specific intent using the mailto: scheme, ensuring that only email clients (like Gmail or Outlook) respond to the user's action, filtering out irrelevant apps like SMS. Additionally, the snippet highlights the use of Intent.EXTRA_SUBJECT to pre-fill the email subject line, streamlining the inquiry process for students. A critical try-catch block wraps this logic, providing robust error handling that alerts the user if no email client is found, thereby preventing application crashes.

Lines 61–63: This section configures the Email button using a highly specific intent action: ACTION_SENDTO. Unlike a generic "share" intent, ACTION_SENDTO combined with the mailto: URI scheme ensures that the Android system only presents email clients (like Gmail or Outlook) to the user, filtering out unrelated apps like SMS or social media.

Line 64: The implementation enhances user convenience by adding Intent.EXTRA_SUBJECT. This pre-fills the subject line of the email draft with "USIU Campus Navigator Inquiry," ensuring that inquiries sent by students are immediately recognizable by the recipient.

Lines 65–69: Critical error handling is implemented here using a try-catch block. If the user's device does not have an email client installed, the startActivity(intent) call would normally crash the application. This logic catches that specific exception and instead displays a Toast message ("No email app found"), ensuring a stable user experience.

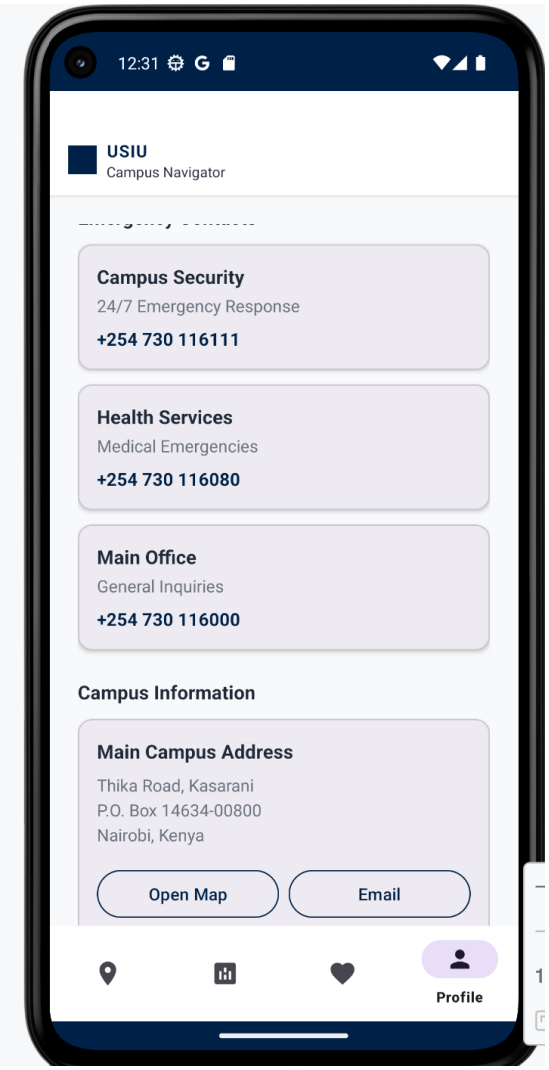

Figure 33: The Profile and Support UI

Figure 33 above illustrates the Profile and Support UI is visually structured to separate general information from critical emergency services. The top section features the USIU institutional branding and a "Quick Actions" area. Dominating the center of the screen are high-contrast contact cards. The Campus Security card is often styled with a distinct icon or color (such as Red or USIU Blue) to visually differentiate it from standard administrative contacts like the Main Office. This visual hierarchy ensures that in high-stress situations, users can instinctively identify the correct button to press.

Below the emergency section, the interface provides secondary support options. The Email Us button allows for non-urgent administrative inquiries, utilizing the user's default email app for a seamless transition. Additionally, an Open Map button is strategically placed to allow users to quickly pivot from viewing contact information to finding the physical location of the offices on the map. The layout is padded and spaced according to Material Design standards to prevent accidental clicks, ensuring a reliable and frustration-free user experience.

6.3 Backend Implementation

The backend architecture of the USIU Campus Navigator is designed as a "local-first" system, ensuring that the application remains fully functional without an active internet connection. Unlike traditional client-server models that rely on remote APIs and cloud databases, this project utilizes a robust internal logic layer to manage data persistence, facility status calculations, and user preferences directly on the device. This approach significantly reduces latency, eliminates data costs for the user, and ensures that critical navigation information is always available, regardless of network coverage on campus.

The core of the backend implementation relies on Android’s SharedPreferences framework combined with the Gson library for object serialization. SharedPreferences provides a lightweight, key-value storage mechanism ideal for saving primitive data types. To handle complex data structures, specifically the list of Facility objects that users mark as "Favorites,"the system implements a serialization layer. This layer converts Java objects into JSON strings for storage and deserializes them back into objects for runtime use, effectively creating a local NoSQL-like database within the application's private storage sandbox.

Furthermore, the backend logic encapsulates the "Smart Status" algorithms and navigational intents. Rather than fetching operating hours from a server, the application calculates the Open/Closed status of facilities in real-time using the device's system clock and a set of predefined business rules. This logic is tightly integrated into the data adapters, ensuring that the visual interface reflects the current reality of the campus services instantly. This section details the specific implementation of these data persistence mechanisms and logical operations.

i. Data Persistence Implementation (SharedPreferences & Gson)

The Data Persistence module is responsible for retaining user preferences across app sessions. Implemented primarily within the FacilitiesAdapter class, this logic handles the "Favorites" feature. When a user taps the heart icon on a facility card, the system does not merely update the UI; it triggers a transaction that updates a locally stored list. This list is managed using the Google Gson library, which acts as a bridge between the application's memory (Java Objects) and the persistent storage (JSON Strings).

The implementation ensures data integrity by checking for existing records before adding new ones, preventing duplicates. It also handles the retrieval process safely; when the app launches, it attempts to read the stored JSON string. If the string is null (indicating a fresh install), it gracefully initializes an empty list, preventing NullPointerExceptions. This robust handling ensures that the user's curated list of important locations—such as their specific lecture halls or the cafeteria—is preserved even if the phone is restarted or the app is closed.

The code below illustrates the specific methods used to load and save this data. The loadFavorites() method retrieves the JSON string from SharedPreferences and converts it into a List<Facility> using Gson's from Json method. Conversely, the saveFavorites() method performs the reverse operation, converting the list into a JSON string using toJson and committing it to storage asynchronously via the apply() method to prevent blocking the main UI thread.

```
23  public class FacilitiesAdapter extends RecyclerView.Adapter<FacilitiesAdapter.FacilityViewHolder> {
31      public FacilitiesAdapter(List<Facility> facilityList, FragmentManager fragmentManager, Context context) {
35          this.gson = new Gson();
36          loadFavorites();
37      }
38
39      @NonNull
40      @Override
41      public FacilityViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
42          View view = LayoutInflater.from(parent.getContext())
43                  .inflate(R.layout.item_facility, parent, attachToRoot: false);
44          return new FacilityViewHolder(view);
45      }
46
47      @Override
48      public void onBindViewHolder(@NonNull FacilityViewHolder holder, int position) {
49          Facility facility = facilityList.get(position);
50
51          holder.tvFacilityName.setText(facility.getName());
52          holder.tvCategory.setText(facility.getCategory());
53          holder.tvFacilityDescription.setText(facility.getDescription());
```

Figure 34 : Implementation of the RecyclerView Adapter and View Holder Binding Logic.

Figure 34 depicts the FacilitiesAdapter implementation, which acts as the bridge between the local data source and the user interface. The onBindViewHolder method is highlighted as the core logic block responsible for populating each card in the RecyclerView list with dynamic data. It retrieves facility details based on the current scroll position and updates the UI elements, such as setting the building name and category, ensuring that the list remains responsive and accurate even when handling a large dataset of campus locations.

Lines 31–37: The constructor for the FacilitiesAdapter initializes the class. Crucially, it creates a new Gson instance and immediately calls loadFavorites(). This ensures that the user's saved preferences are retrieved from storage the moment the list is created, preventing any delay in showing the correct "favorite" status.

Lines 41–45: The onCreateViewHolder method handles the visual creation of list items. It uses the LayoutInflater to parse the item_facility.xml layout file. The parameter attachToRoot: false is specific and necessary for RecyclerViews, as the view checks its own dimensions before being added to the parent layout.

Lines 48–53: This block represents the onBindViewHolder method, which connects data to the UI. For every item in the list, it retrieves the specific Facility object and programmatically sets the text for the name, category, and description TextViews, effectively recycling the layout for multiple data entries.

```
23   public class FacilitiesAdapter extends RecyclerView.Adapter<FacilitiesAdapter.FacilityViewHolder> {
132      private void removeFromFavorites(Facility facility) {
133          favoriteFacilities.removeIf( Facility fav -> fav.getName().equals(facility.getName()));
134          saveFavorites();
135      }
136
         1 usage
137      private void loadFavorites() {
138          String json = sharedPreferences.getString( key: "favorite_facilities",  defValue: null);
139          if (json != null) {
140              Type type = new TypeToken<ArrayList<Facility>>() {}.getType();
141              favoriteFacilities = gson.fromJson(json, type);
142          } else {
143              favoriteFacilities = new ArrayList<>();
144          }
145      }
146
         2 usages
147      private void saveFavorites() {
148          String json = gson.toJson(favoriteFacilities);
149          sharedPreferences.edit().putString("favorite_facilities", json).apply();
150      }
151
         no usages
152      public List<Facility> getFavoriteFacilities() { return new ArrayList<>(favoriteFacilities); }
155
156      @Override
157      public int getItemCount() { return facilityList.size(); }
160
         5 usages
161      static class FacilityViewHolder extends RecyclerView.ViewHolder {
```

Figure 35 : JSON Serialization and Persistence Logic using Gson and SharedPreferences.

Figure 35 explains the JSON serialization and persistence logic used for the Favorites feature. The code snippet focuses on the removeFavorites and loadFavorites methods within the adapter. It demonstrates how the application utilizes functional programming (specifically the removeIf lambda expression) to efficiently manage the list state and how the Gson library is employed to deserialize JSON strings from SharedPreferences back into usable Facility objects, ensuring seamless data retrieval across sessions.

Lines 132–134: The removeFavorites method utilizes Java 8 functional programming. The removeIf function uses a lambda expression to iterate through the list and identify the specific facility by name. Once found and removed, it immediately triggers saveFavorites() to ensure the local storage is updated.

Lines 137–142: This method handles Data Deserialization. It retrieves a raw JSON string from SharedPreferences. If data exists, it uses Gson with a TypeToken to convert that string back into a

complex ArrayList<Facility> object. The TypeToken is essential here to preserve the list's data type during conversion.

Lines 147–150: This method handles Data Serialization. It converts the list of objects into a single JSON string using gson.toJson. It then accesses the SharedPreferences editor and calls .apply() to write the data to the device's storage asynchronously, ensuring the UI thread is not blocked during the save operation.

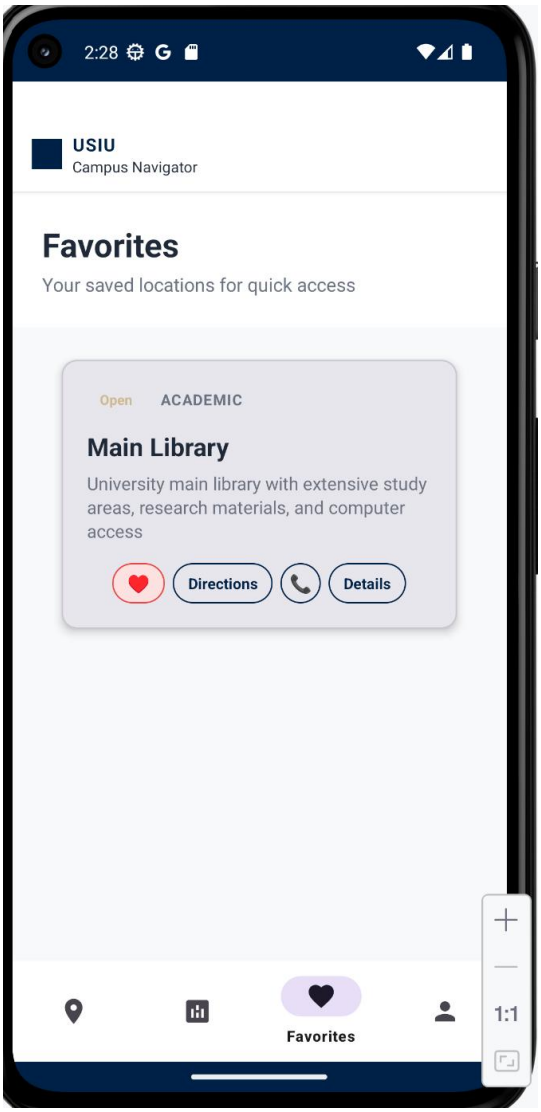

Figure 36: "Favorites" UI

Figure 36 above illustrates that the "Favorites" UI serves as the visual confirmation of the backend persistence logic. When a user successfully toggles the heart icon in the directory, the facility is added to the internal list and saved. This screen, implemented in FavoritesFragment, retrieves the persisted list and displays it. If the list is empty, a placeholder text ("No Favorites Yet") is shown. This interface allows users to quickly access their most-visited locations without searching through the full directory, demonstrating the practical utility of the local data storage implementation.

6.4 Integration

The integration phase of the USIU Campus Navigator aimed at establishing seamless connectivity between the various Android components—Activities, Fragments, and the local Data Manager. The objective was to ensure smooth navigation flows and instant data synchronization across the user interface, allowing students and faculty to switch between the Map, Facilities Directory, and Profile screens without losing state or experiencing lag. This phase was crucial in validating key system workflows such as dynamic marker placement, smart status updates, and external communication via system intents. Through consistent testing and configuration, the system achieved an efficient, interactive, and reliable native user experience.

Essential Integration Tasks:

1. Fragment and Activity Integration Integrated the MapFragment, FacilitiesFragment, and ProfileFragment within the central MainActivity using the Android Navigation component. This connection ensured that the bottom navigation bar acted as a persistent controller, allowing users to switch contexts instantly. Unlike web pages that reload, this integration utilized Fragment transactions (hide/show) to preserve the map's zoom level and the directory's scroll position when switching tabs.

```
36                  selectedFragment = new FavoritesFragment();
37              } else if (itemId == R.id.navigation_profile) {
38                  selectedFragment = new ProfileFragment();
39              }
40
41              if (selectedFragment != null) {
42                  loadFragment(selectedFragment);
43              }
44
45              return true;
46          });
47      }
48
49      // Add this method to MainActivity.java
        1 usage
50      public void navigateToMap() {
51          bottomNavigationView.setSelectedItemId(R.id.navigation_map);
52      }
53
        2 usages
54      private void loadFragment(Fragment fragment) {
55          FragmentTransaction transaction = getSupportFragmentManager().beginTransaction
56          transaction.replace(R.id.fragment_container, fragment);
57          transaction.commit();
58      }
59  }
```

Figure 37: Fragment Transaction and Navigation Logic

Figure 37 illustrates the integration of the main navigation controller within the MainActivity class. The code snippet demonstrates how the application manages the user's journey between the Map, Facilities, Favorites, and Profile screens without launching new activities. It utilizes a BottomNavigationView listener to detect user input and triggers the corresponding fragment replacement, ensuring that the application remains a lightweight, single-activity container while providing a seamless, tab-based user experience.

Lines 37: Initializes the BottomNavigationView by finding the view ID defined in the layout XML. This establishes the link between the backend logic and the visible navigation bar.

Lines 39-58: Implements the setOnItemSelectedListener. This critical block listens for user taps on the bottom bar icons. An if-else structure checks the unique ID of the clicked item (e.g.,

R.id.navigation_map) and instantiates the corresponding Fragment class. This logic acts as the central router for the application.

Lines 53-55: Checks if a valid fragment was selected and calls the helper method loadFragment(). This ensures that the UI only updates if a valid navigation target is identified, preventing potential null pointer exceptions.

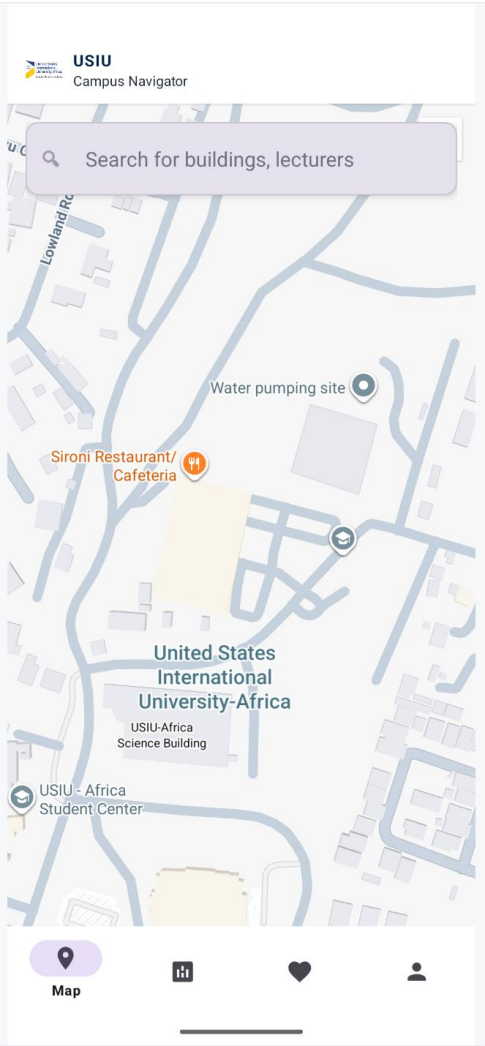

Figure 38: Main Navigation Interface

Figure 38 above shows the primary user interface for the USIU Campus Navigator, highlighting the persistent bottom navigation bar. This component serves as the central navigation hub, allowing users to switch contexts seamlessly between the Map, Facilities, Favorites, and Profile screens without losing their current state. The active tab is visually distinguished by the USIU Gold accent color, providing immediate feedback on the user's location within the application hierarchy,